Practica Final:

Sistema de recomendación de películas

Metodología de la Programación

1º Ingeniería Informática

Grupo: E

Curso 2025/2026

Índice

1. Objetivos **2**

2. Cálculos a realizar en la práctica **3**

3. Acciones de mejora sobre las clases existentes **6**
3.1. Clase Usuario 6
3.2. Clase ConjuntoUsuarios 6
3.3. La clase ConjuntoPeliculas 7

4. La clase Opiniones **9**
4.1. Motivación 9
4.2. Variables y métodos 10

5. Main.cpp y estructura de ficheros **14**
5.1. Estructura de main.cpp 15
5.2. Estructura de los ficheros del sistema de recomendaciones 17

6. Detalles a tener en cuenta **19**

7. Ejemplo de ejecución **20**
7.1. Generación de recomendaciones de películas 20
7.2. Estudio de la valoración de una película 20
7.3. Modificación de una opinión de usuario 21

8. Entrega **21**

1. Objetivos

En esta práctica final, utilizaremos las clases desarrolladas en las prácticas 2 y 4 para la creación de todo el código necesario para la creación de un sistema de recomendación de películas basado en la red social de películas que ya hemos creado. Nótese que no todas las funciones realizadas en las prácticas 2 y 4 serán de utilidad para la práctica final. Por tanto, se permite la entrega con únicamente los métodos de dichas clases que sean necesarios para la ejecución del programa.

En las prácticas 2 y 4 hemos manejado varias clases que nos permitían el manejo de conjuntos de películas y usuarios. En esta práctica crearemos una nueva clase, la clase Opiniones, que representará las valoraciones que los usuarios han proporcionado al sistema sobre las películas. Para ellos crearemos una matriz dinámica $n \cdot m$ donde n es el número de usuarios y m el número de películas. Además, implementaremos métodos que nos permitirán recomendar películas no vistas a los usuarios en función de las películas que han visto los usuarios más afines a ellos. Finalmente, calcularemos la valoración media que los usuarios dan a las películas y si éstos están de acuerdo en la valoración o no, esto es, determinaremos si los usuarios tienen opiniones muy diferentes de las películas que han visto.

El programa que se ha de implementar almacenará toda la información en tres ficheros diferentes. En cuanto a las películas y usuarios, se seguirá el mismo formato que el visto en las prácticas 2 y 4. Finalmente, se creará un tercer ficheros que almacenará la información acerca de las opiniones de los usuarios introducidas al sistema.

Cada ejecución del programa llevará a cabo una de las siguientes acciones:

- **Modificación de las opiniones de un usuario sobre una película**: Un usuario puede modificar una opinión que tenga de una película. En concreto, esta acción puede acabar realizando una de las tres siguientes operaciones:
  - *Eliminación de la opinión proporcionada para una película*: El usuario elimina su opinión de la película. La película pasa a ser indicada como no vista y se reduce en uno el número de películas vistas asignado al usuario.
  - *Modificación de la opinión proporcionada*: El usuario se lo ha pensado mejor y modifica la opinión que tenía sobre una película.
  - *Agregación de una opinión nueva*: El usuario ha visto una nueva película, la puntúa y aumenta en 1 el número de películas vistas.

  Una vez realizada la modificación, los ficheros donde se almacena la información se modifican y

quedan actualizados.

- **Recomendación de películas**: El sistema recomienda una serie de películas a un determinado usuario. Para ello, se busca a los usuarios a los que les hayan gustado películas parecidas a las del usuario. Una vez encontrados los usuarios más afines, se buscan las películas que el usuario no haya visto. Esas son las películas que podrían gustarle.

- **Valoración de una película**: El sistema muestra la valoración media de una película, el consenso alcanzado en la valoración y el consenso general de los usuarios que hay incluidos en el sistema.

Todo este esquema de ejecución que hemos comentado se debe de montar sobre las clases que se han implementado en las prácticas 2 y 4. Esto es, las clases Usuario, ConjuntoUsuarios, Pelicula y ConjuntoPeliculas. Sobre dichas clases, tendremos que introducir una serie de mejoras y métodos adicionales. Todas las acciones mínimas necesarias de mejora están descritas en la sección 3. Además, como ya hemos comentado, crearemos una clase nueva cuya principal función será almacenar las opiniones de los usuarios sobre las películas y llevar a cabo cálculos sobre dicha información. Su especificación se describe en detalle en la sección 4.

2. Cálculos a realizar en la práctica

Tal y como hemos comentado en la sección anterior, para la práctica final implementaremos un sistema de recomendaciones utilizando como base las clases implementadas en las prácticas 2 y 4. En esta sección explicaremos en detalle las operaciones que hay que realizar para calcular todos los valores que se piden en la práctica. Para mayor claridad, explicaremos todos los cálculos tomando como ejemplo la información de la Tabla 1. Tal y como podemos observar, esta tabla es una tabla de usuarios por películas donde, en cada casilla, se indica si el usuario ha visto la película y, en caso de haberla visto, la calificación en escala [0,10] que le da. Para indicar que un usuario no ha visto una película se utiliza el valor -1. Por último, mostramos en negrita en la tabla los ids de las películas y de los usuarios. En total tenemos 5 películas y 6 usuarios en este ejemplo.

	0	**1**	**2**	**3**	**4**
0	9	-1	-1	8	7
1	3	-1	2	4	2
2	9	7	9	8	8
3	9	8	-1	-1	-1
4	8	-1	-1	6	4
5	3	2	-1	2	3

Tabla 1: Ejemplo de matriz de opiniones.

A continuación, detallamos cómo calcular cada una de las medidas que necesitamos implementar

para crear el sistema de recomendaciones:

- **Media de la valoración de las películas**: Hacemos la media, para cada película, de los valores proporcionados por los usuarios. Esto es, se hace la media de aquellos valores de una columna que no sean -1. Por ejemplo, para las películas con id 1 y 2 de la tabla 1, los resultados serían los siguientes:

$$media(id1) = \frac{7+8+2}{3} = 5,66$$
$$media(id2) = \frac{2+9}{2} = 5,5$$

- **Consenso para una determinada película**: La medida del consenso para una determinada película indica el nivel de acuerdo alcanzado entre los usuarios. Es un valor que se encuentra en el intervalo [0,1] indicando 0 un total desacuerdo entre los usuarios y 1 un total acuerdo en la puntuación dada a la película. Este valor se calcula haciendo la media a las diferencias de puntuación dadas por los usuarios a las películas. Por ejemplo, para la película con id 1, los usuarios 2, 3 y 5 han proporcionado, respectivamente, las valoraciones 7, 8 y 2. Por tanto, para calcular el consenso, primero calculamos las diferencias entre las puntuaciones dadas por los usuarios. Por tanto, al tener 3 usuarios, calculamos la diferencia entre el usuario 2 y el 3, el usuario 2 y 5 y, finalmente, la diferencia entre las puntuaciones de los usuarios 3 y 5. Los resultados pueden observarse en la Tabla 2. Para calcular este valor de diferencia utilizamos la siguiente expresión:

$$distancia(u_1, u_2) = \frac{|u_1 - u_2|}{10} \tag{1}$$

donde la distancia entre las opiniones del usuario 1 y 2 es el valor absoluto de la diferencia. La aplicación del valor absoluto permite que el valor obtenido siempre sea positivo. Para la implementación de este cálculo, podemos hacer uso de la función abs de la biblioteca math. El valor obtenido se divide entre 10 para poder obtener un valor entre 0 y 1 que es un intervalo más apropiado para indicar porcentajes. Si el alumno así lo prefiere, puede multiplicar por 10 y así obtener el valor de consenso en calidad de porcentaje (intervalo [0,100]).

Usuarios	**Cálculo**	**Resultado**
2 y 3	(|7-8|)/10	0,1
2 y 5	(|7-2|)/10	0,5
3 y 5	(|8-2|)/10	0,6
Consenso	(0,1+0,5+0,6)/3	**0,4**

Tabla 2: Cálculo del consenso para una de las películas.

Una vez calculados todos los posibles valores de distancia entre aquellos usuarios que hayan proporcionado información sobre la película, simplemente hacemos la media sobre los valores obtenidos. De esta forma obtenemos el valor de consenso.

- **Consenso general**: Para calcular el consenso general, calculamos el consenso para cada una de las películas y luego hacemos la media. De esta manera obtenemos un valor que indica el porcentaje medio de acuerdo que hay en las opiniones de las películas para todos los usuarios del sistema de recomendación.

- **Similitud entre usuarios**: Dos usuarios serán similares entre sí si, para todas las películas que hayan visto, la puntuación que han dado a las películas es similar. Para ello, aplicaremos una métrica muy similar a la usada para calcular el consenso de una película. Veamos, por ejemplo, como calcularíamos la similitud entre los usuarios con id 0 y 1. Lo primero, es ver qué películas han visto ambos. En este caso, las películas que tienen en común son la 0, la 3 y la 4. A continuación, tenemos que ver la diferencia de puntuación en cada una de las películas y luego hacer la media de los valores obtenidos. Para ver la cercanía de puntuación de los usuarios en una película concreta utilizaremos una expresión análoga a la expresión (1). Podemos ver el cálculo de la similitud entre los usuarios 0 y 1 en la tabla 3. Las películas en las que alguno de los dos usuarios o los dos no hayan introducido la puntuación no se utilizan ya que ahí no tendría sentido estudiar la similitud de las puntuaciones. El valor de similitud entre dos usuarios es un valor localizado en el intervalo [0,1]. Cuanto más cercano sea un valor obtenido a 0, más similares son ambos usuarios. Por el contrario, mientras más cercano a 1 sea el valor obtenido, más diferencias tendrán.

Películas	**Cálculo**	**Resultado**
0	(|9-3|)/10	0,6
3	(|8-4|)/10	0,4
4	(|7-2|)/10	0,5
Similitud	(0,6+0,4+0,5)/3	0,5

Tabla 3: Cálculo de la similitud de los usuarios con ids 0 y 1.

- **Creación de la lista de películas recomendadas**: Para la creación de la lista de películas recomendadas para un usuario concreto debemos de seguir los siguientes pasos:

  - Calculamos la similitud de dicho usuario con todos los demás usuarios. Ordenamos a todos los demás usuarios en función de su similitud con el usuario al que queremos hacer la recomendación.

  - Seleccionamos los x usuarios más similares al usuario al que queremos generar la recomendación. x, como veremos en la sección 4, es un valor que tendrá que ser proporcionado. Cuanto mayor sea x más usuarios tendremos en cuenta en la recomendación y más películas serán recomendadas al usuario.

  - Para los usuarios seleccionados, analizamos las películas que han visto y seleccionamos aquellas que el usuario para el que estamos generando la recomendación no haya visto y que, además, hayan sido puntuadas con buena nota.

- Por último, ordenamos las películas según su puntuación IMDB y las mostramos al usuario. Esas son las películas que, no habiendo sido vistas por el usuario, han sido vistas por usuarios con gustos similares a los suyos y han obtenido una buena puntuación.

Veamos un ejemplo de los dos últimos puntos del proceso. Imaginemos que queremos generar una recomendación al usuario 0 a partir de las películas vistas por los usuarios 1 y 2. La única película que el usuario 1 ha visto y que el usuario 0 no es la película con id 2 (esto es porque en la posición [0][2] hay un -1 y en la posición [1][2] hay un 2. Sin embargo, dado que el usuario 1 ha puntuado de forma bastante negativa la película 2, ésta no sería recomendable para el usuario 0. Como esa es la única película que el usuario 1 ha visto pero el 0 no entonces no podemos generar recomendaciones. Con respecto al usuario 2, las películas que él ha visto pero el usuario 0 no son las películas 1 y 2. Como este usuario les ha dado buena nota a esas 2 películas, entonces recomendaríamos al usuario 0 que viera las películas 1 y 2.

3. Acciones de mejora sobre las clases existentes

En esta sección pasaremos a describir, para cada una de las clases que ya se han implementado en prácticas previas, qué cambios y modificaciones debemos llevar a cabo. Las modificaciones descritas son las modificaciones mínimas que la creación del sistema de recomendación requiere. Por tanto, el alumno puede introducir todos aquellos cambios extra que le ayuden a la creación del sistema planteado.

3.1. Clase Usuario

La clase Usuario, al contener únicamente información de cada uno de los usuarios del sistema, no requiere grandes modificaciones. El sistema de recomendación que vamos a crear permite al usuario retirar opiniones sobre películas del sistema. La variable de la clase numpeliculas irá ahora asociada al número de películas sobre las que el usuario ha opinado. Por tanto, dentro de la clase Usuario, puede ser interesante incluir un método **void decrementaNumPeliculas()** que decremente el número de películas asociado al usuario. De esta forma, llamaremos al método cada vez que el usuario decida retirar una opinión del sistema, lo que hará que la variable numpeliculas del usuario sea consistente.

3.2. Clase ConjuntoUsuarios

La clase ConjuntoUsuarios es la que almacena el conjunto de usuarios que forman parte del sistema. Esta clase tampoco va a requerir grandes modificaciones ya que los cambios que se proponen van más enfocados a obtener un acceso más sencillo a los datos de la clase con el objetivo de facilitar la implementación de los métodos de la clase Opiniones. A continuación, se listan los cambios propuestos:

- **Operador » de la clase**: El sistema de recomendación, como ya hemos comentado, almacena en un fichero los datos de todos los usuarios del sistema. Esto permite la posterior recuperación y uso de la información en ejecuciones posteriores del programa. Por tanto, debemos incluir un método u

operador que nos permita leer los datos de los usuarios de un fichero y cargarlos en un objeto de la clase. Se hablará en más profundidad acerca del formato de los ficheros en la sección 5.2.

- **Ordenación por id**: Para facilitar la implementación del sistema, vamos a hacer que el id de los usuarios coincida con su posición en la matriz. Por tanto, si hay 5 usuarios, estos deben ser introducidos siendo el primero el usuario con id 0, el segundo el que tenga id 1 y así hasta el 5º usuario que tendrá id 4. La ejecución del programa requerirá que se le indique un fichero de donde leer los valores de los usuarios. Es importante que el fichero tenga los ids colocados de forma correcta o que estos se asignen automáticamente. Al leer los usuarios de un fichero, no habrá opción en el programa a la agregación de nuevos usuarios aparte de los indicados en el fichero. Por tanto, ya no es necesario ordenar por id el conjunto. Es más, esto puede ser contraproducente para la creación de algunas de las funciones pedidas, como la que calcula los usuarios más similares a uno dado ya que interesará más que estos estén ordenados por similitud al usuario en cuestión y no por id.

- **Método clear()**: Este método elimina toda la información que contuviera el objeto ConjuntoUsuarios y lo deja preparado para la inclusión de nueva información. El vector de Usuarios queda inicializado teniendo en cuenta los siguientes valores: numusuarios=0 y tamreservado=INCREMENTO.

- **Método int numusuarios() const**: La creación del sistema de recomendación va a requerir operaciones más complejas que las de agregar y eliminar usuarios. Por tanto, para poder crear bucles sobre los usuarios de un conjunto de usuarios, vamos a requerir un método que nos devuelva el número de usuarios que hay en el objeto ConjuntoUsuarios.

3.3. La clase ConjuntoPeliculas

En la práctica 2, implementamos la clase ConjuntoPeliculas como un vector estático de películas. Sin embargo, ya hemos visto como esto resulta ineficiente e impide que podamos almacenar grandes cantidades de películas en el vector. Por tanto, para esta práctica, se pide modificar la clase de forma que los objetos película se almacenen en un vector reservado en con memoria dinámica. Con el objetivo de facilitar la búsqueda de errores y la comprensión de la práctica, los ficheros de películas y usuarios propuestos no tendrán grandes cantidades de datos sino que serán ficheros de prueba que nos permitirán fácilmente determinar si el programa funciona adecuadamente o no. Sin embargo, queremos que el programa implementado funcione adecuadamente independientemente del número de elementos que haya tanto de usuarios como de películas. Para adecuar esta clase al sistema de recomendación pedido, se piden las siguientes modificaciones:

- **Modificación de variable principal, constructores y destructores**: La variable que almacena ahora la información debe de ser de tipo Pelicula *. Los constructores y destructores de la clase deben de adaptarse para trabajar con el nuevo tipo de dato de forma que reserven y liberen la memoria según sea necesario. Esto incluye el constructor de copia. Para facilitar el uso de la clase, también pueden incluirse en ConjuntoPeliculas, todos los operadores que se han propuesto para la clase ConjuntoUsuarios.

- **Operadores = y +=**: De entre los operadores propuestos y que pueden ser implementados, será interesante, por la comodidad que nos dará a la hora de programar, implementar el operador de asignación y el +=. El contenido de ambas implementaciones serán análogas a las realizadas en ConjuntoUsuarios.

- **Uso de la variable INCREMENTO**: Al igual que ocurría en la clase ConjuntoUsuarios, tendremos una constante INCREMENTO que indica cuantas posiciones nuevas debemos de reservar una vez que el vector de películas esté lleno. Inicialmente, se reservarán INCREMENTO posiciones en el vector.

- **Ids de los objetos película**: De forma análoga a lo que comentamos de los ids de los usuarios, los ids de las películas serán consecutivos y comenzarán desde 0. De esta forma, los valores i y j de la matriz usuarios x peliculas que crearemos en el objeto Opiniones correponderán con el usuario i y la película j facilitando mucho la implementación del sistema de recomendación propuesto.

- **Método int numpeliculas() const**: Este método devuelve el número de películas que contiene el objeto de la clase. La inclusión de este método en la clase nos ayudará con la implementación de los métodos que se piden para generar las recomendaciones y que formarán parte de la clase Opiniones.

- **Método int busquedaPelicula(int busquedaid)**: Este método recorre el vector de películas y devuelve la posición de la película que tiene un id específico. Para facilitar el acceso directo a la película, también se puede implementar el operador [] de forma que se devuelva la película en vez de la posición de la película en el vector. Sino, puede implementarse este método y luego usarse el método at ya implementado en la clase.

- **Método int busquedaPelicula(string nombrepelicula)**: En la llamada al programa de recomendación de películas, utilizaremos el nombre de la película en vez del id ya que esta es la información que los usuarios conocen sobre las películas. Por tanto, necesitamos un método que, a partir de un nombre de película, nos proporcione su id. Recomendamos utilizar en este método el método find de la clase string ya que permite la búsqueda de substrings dentro de un string. Esto permitirá que, proporcionando sólo parte del título, el método sea capaz de encontrar la película y devolver su id. Podéis encontrar más información de como funciona el método find así como ejemplos de uso en el siguiente enlace: `https://cplusplus.com/reference/string/string/find/`. En el caso de haber dos películas que cuadren con el string introducido en la variable nombrepelícula, en aras de la simplicidad, se permite que simplemente se devuelva el id de la primera película encontrada.

Gracias a las modificaciones propuestas para esta clase podremos tener un acceso más cómodo a los elementos de la clase ConjuntoPeliculas así como la capacidad de poder representar grandes cantidades de películas de forma sencilla usando memoria dinámica.

4. La clase Opiniones

4.1. Motivación

Hasta ahora, hemos desarrollado clases que permiten la representación de usuarios y películas en el sistema. Sin embargo, no hemos implementado ni la lógica referente a las opiniones de los usuarios sobre las películas ni las funciones de recomendación. Esta lógica la implementaremos en esta práctica final utilizando una nueva clase, la clase Opiniones.

La clase Opiniones estará formada principalmente por tres objetos. Por un lado, tendremos un objeto de la clase ConjuntoUsuarios, en donde se almacenará la información referente a los usuarios del sistema de recomendación. Por otro lado, tendremos un objeto de la clase ConjuntoPeliculas, en donde almacenaremos las películas sobre las que los usuarios del sistema pueden votar así como todas sus características. Por último, dispondremos de una matriz de flotantes dinámica cuyo tamaño vendrá dado por el número de elementos de los objetos ConjuntoUsuarios y ConjuntoPeliculas definidos. Por tanto, nos encontramos con una matriz de tamaño numUsuarios x numPeliculas en donde, en la posición [i][j] se almacena la opinión que tiene el usuario i sobre la película j. Para facilitar la implementación, como ya hemos comentado, haremos que las posiciones en la matriz se correspondan con los ids de los elementos de los conjuntos de usuarios y películas. En caso de que el usuario i no haya visto la película j, entonces en la posición [i][j] de la matriz colocaremos un valor -1. Fuera de la clase Opiniones pero dentro del .h y .cpp donde se define la clase implementaremos tres funciones:

- generaRecomendacionUsuario.
- calculaConsensoPelicula.
- cambiaOpinion.

Cada una de ella cumplirá con una de las acciones comentadas en la sección 1. Esto es, generar un listado de películas a recomendar para un usuario concreto, obtener información de la votación de una película específica o modificar la opinión de un usuario sobre una película respectivamente.

La clase Opiniones es la clase principal que debe configurarse desde el main para el correcto funcionamiento del sistema de recomendaciones. Como ya hemos visto, este objeto contiene instancias de objetos de las otras clases ya implementadas. Concretamente, puede verse el esquema de inclusión de objetos de clases en la Figura 1.

En la siguiente subsección, veremos una descripción detallada de cada una de las variables, funciones y métodos relacionados con la clase Opiniones.

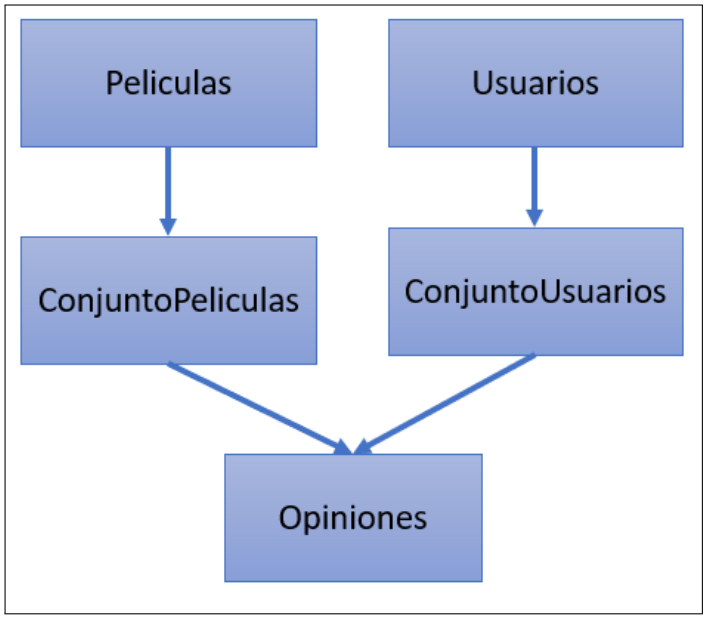

Figura 1: Relación de uso entre las diferentes clases definidas.

4.2. Variables y métodos

A continuación, pasaremos a describir las variables, métodos y funciones que deben de aparecer en los archivos Opiniones.h y Opiniones.cpp. Las funciones aparecerán fuera de la clase Opiniones mientras que los métodos aparecerán dentro de la clase. Puede verse un ejemplo de como hacer este tipo de implementación en la Práctica 1.

Las variables que conforman la clase Opiniones son:

- **ConjuntoUsuarios users**: Conjunto de usuarios inscritos en el sistema de recomendación.
- **ConjuntoPeliculas pelis**: Conjunto de películas disponibles para su votación por parte de los usuarios.
- **float ** opiniones**: Matriz que almacena la información de las valoraciones que los usuarios han dado a las películas. Si un usuario no ha votado una película, se indica con el valor -1.

Nótese que para no replicar información y evitar posibles inconsistencias, se recomienda no definir ninguna variable tipo int filas o int columnas. Por tanto, la reserva de memoria del objeto opiniones puede hacerse utilizando los métodos numpeliculas() y numusuarios() de los objetos users y pelis.
Los métodos que conforman la clase Opiniones son:

- **Opiniones(ConjuntoUsuarios newusers, ConjuntoPeliculas newpelis)**: Crea un nuevo objeto opi-

niones utilizando los usuarios y películas que se le pasan como parámetro. La matriz de opiniones se inicializa de acuerdo al tamaño de los conjuntos y todos los valores se inicializan a -1 indicando que ningún usuario ha introducido ninguna opinión.

- **Opiniones(string fichopiniones, string fichpelis, string fichusuarios)**: Crea un nuevo objeto opiniones y carga un conjunto de usuarios, un conjunto de películas y una matriz de opiniones ya existentes a partir de los ficheros indicados. La información requerida está almacenada en tres ficheros distintos. Su estructura se detallará en la sección 5.2. Este método utiliza los métodos y operadores que permiten leer datos de las clases ConjuntoPeliculas, ConjuntoUsuarios y Opiniones para extraer la información de los ficheros. El resultado es un objeto que contiene toda la información de los ficheros. Para cargar los datos de la matriz de opiniones desde el fichero indicado se puede llamar al método cargaOpiniones descrito más abajo.

- **void saveData(string fichopiniones,string fichpelis, string fichusuarios)**: Este método genera tres ficheros distintos. El primero contiene la información de la matriz de opiniones, el segundo la información de las películas y el tercero la información de todos los usuarios adscritos al sistema. Su estructura será comentada en más profundidad en la sección 5.2. Este método debe almacenar en ficheros toda la información de forma que posteriormente podamos recuperarla y analizarla en posteriores ejecuciones del programa.

- **void cargaOpiniones(string fichopiniones)**: Este método lee del fichero indicado como parámetro la información de la matriz de opiniones y almacena toda la información en la variable.

- **void introduceOpinion(int iduser, int idpelicula, float valor)**: Añade o modifica la opinión que el usuario con id iduser tenía sobre la película con id idpelicula. Si es una nueva opinión, es decir, el valor de la matriz de opiniones para esa posición era -1, entonces hay que incrementar en 1 el número de películas visto por el usuario. En caso contrario, únicamente se modifica el valor de la opinión ya que consideramos que el usuario cambia de opinión y modifica la opinión previamente introducida. Por tanto, en este caso, el número de películas vistas no cambia.

- **void eliminaOpinion(int iduser, int idpelicula)**: Se elimina la opinión que el usuario iduser había proporcionado para la película idpelicula. Además, se decrementa el número de películas vistas en 1. Este método debe comprobar que efectivamente el usuario había proporcionado una valoración a la película. En caso de no ser así, el método no hace nada.

- **string to_string() const**: Genera un string con la información de la matriz opiniones. La estructura será la típica en estos casos: añadimos el carácter \n al final de cada fila y dejamos un espacio entre los diferentes elementos. Puede verse un ejemplo de como quedaría la matriz en la Figura 2 donde el string resultante sería el encargado de dibujar, de entre todo el texto generado por el operador «, la matriz de opiniones.

- **Opiniones(const Opiniones & orig)**: Constructor de copia de la clase Opiniones. Al contener la

clase una matriz dinámica, es necesaria su implementación. Al ser un constructor, es necesario que este reserve memoria para la matriz de opiniones.

- **~Opiniones()**: Destructor de la clase. Al contener la clase una matriz dinámica, es necesaria la liberación de la memoria una vez el objeto deja de ser de utilidad.

- **friend std::ostream & operator« (std::ostream & flujo, const Opiniones & conjop)**: Este operador introduce en el flujo de salida toda la información de la clase Opiniones por pantalla. Esto incluye la información de los conjuntos de usuarios y de películas y la matriz de opiniones. El formato a seguir es el indicado en la Figura 2. Principalmente, utilizaremos este operador para mostrar por pantalla toda la información del objeto de opiniones. Aun así, es posible, si el usuario lo desea, diseñar un operador » que lea la información y modificar el programa para que toda la información se almacene en un único archivo. Es interesante, a la hora de implementar este método, hacer uso de los operadores « ya creados.

- **ConjuntoUsuarios & getusers() const**: Devuelve el objeto users de la clase Opiniones. El objetivo de este método es poder acceder a la información de los usuarios fuera de la clase.

- **ConjuntoPeliculas & getpelis() const**: Devuelve el objeto pelis de la clase Opiniones. El objetivo de este método es poder trabajar con la información de las películas fuera de la clase.

- **ConjuntoPeliculas peliculasrecomendadas(int userid, int numusers)**: Este método llama al método usuariosSimilares con los parámetros userid y numusers para obtener los numusers usuarios más afines al usuarioid. A continuación crea un objeto ConjuntoPeliculas y mete en él todas las películas con puntuación mayor a 6 que los usuarios similares han visto pero él no. Por último, se devuelve el conjunto de películas obtenido ordenado por el ranking IMDB.

- **float similitud(int userid1, int userid2)**: Utiliza las expresiones vistas en la sección 2 para determinar la similitud entre el usuario con id userid1 y el usuario con id userid2. Devuelve un valor flotante con el valor de similitud. En el caso de que dos usuarios no tengan ninguna película en común, entonces el método devuelve -1.

- **ConjuntoUsuarios usuariosSimilares(int userid,int numusers)**: Para todos los usuarios distintos a userid, calcula el valor de similitud. Esto se puede hacer utilizando el método similitud descrito anteriormente. Una vez tenemos la similitud del usuario escogido con todos los demás, cogemos los numusers usuarios más similares y los incluimos en un objeto ConjuntoUsuarios. Finalmente, devolvemos el objeto ConjuntoUsuarios ordenado por valor de similitud con el usuario userid. Si la similitud con alguno de los usuarios es -1 (no tienen películas en común), entonces no se incluye el usuario en el conjunto.

- **float consensoPelicula(int idpelicula)**: Utilizando el proceso explicado en la sección 2, este método calcula el valor de consenso que los usuarios han obtenido para la película con id idpelicula.

- **float consenso()**: Calcula el consenso global para todas las películas. Usamos el método consensoPelicula para calcular el consenso para cada una de las películas del sistema y, una, vez obtenidos los valores, se calcula y se devuelve la media.

- **float mediaPelicula(int idpelicula)**: Calcula el valor medio de opinión que los usuarios tienen sobre la película idpelicula. Para ello, se puede seguir el proceso explicado en la sección 2. En caso de no existir ninguna opinión sobre la película, se devuelve -1.

Nótese que el alumno puede añadir todas aquellas funciones y métodos extras que considere necesarios para la realización de la práctica.

Fuera de la clase Opiniones pero dentro de los ficheros .h y .cpp de dicha clase se definen las siguientes funciones:

- **void generaRecomendacionUsuario(Opiniones op, int idusuario, int numusers)**: Este método utiliza el método peliculasrecomendadas para obtener las películas recomendadas para el usuario idusuario a partir de los numusers usuarios más similares a él. Finalmente, las imprime por pantalla. En la sección 7, podemos ver un ejemplo de qué debería imprimirse por pantalla.

- **void calculaConsensoPelicula(Opiniones op, string nombrepelicula)**: Este método determina el id al que pertenece el nombre de la película que se le ha pasado por parámetro. En caso de no estar esa película en el conjunto de películas, se devuelve un mensaje de error. En caso de sí existir una película con dicho nombre en el conjunto, entonces utilizamos los métodos mediaPelicula, consensoPelicula y consenso y mostramos toda la información por pantalla usando cout. En la sección 7, podemos ver un ejemplo de qué debería imprimirse por pantalla.

- **void cambiaOpinion(int idusuario, string nombrepelicula, float valor, Opiniones op, string fichopiniones, string fichpeliculas,string fichusuarios)**: El objetivo de este método es llevar a cabo el proceso de modificación, eliminación o adición de nuevas opiniones al sistema de recomendación. Este método modifica un valor de opinión del sistema llamando a los métodos introduceOpinion o eliminaOpinion del objeto Opiniones. En el caso de que se indique que valor=-1, entonces se llamará a la función eliminaOpinion. En otro caso, se llamará al método introduceOpinion para agregar o modificar la opinión de un usuario sobre una película. Finalmente, se deben de sobrescribir los ficheros de donde se leyó la información con la modificación hecha. Opcionalmente, se puede mostrar por pantalla la nueva información. También podemos modificar este método para que permita la realización de varios cambios en una misma ejecución del programa. Para ello, el programa pediría el usuario mediante varios cin la información requerida para la modificación de los datos.

Nótese que cada una de estas funciones está relacionada con las posibles acciones que el programa debe poder realizar. En la siguiente sección, se explicará más en detalle las posibles acciones a llevar a cabo según el tipo de llamada que se haga al programa.

```
PELICULAS
0 Thor: Love and Thunder acción 2022 6.400000
1 Top Gun: Maverick acción 2022 8.400000
2 Brahmastra Part One: Shiva acción 2022 5.600000
3 The Woman King acción 2022 6.100000
4 End of the Road acción 2022 4.700000
5 Bullet Train acción 2022 7.500000
6 Moonfall acción 2022 5.100000
7 Everything Everywhere All at Once acción 2022 8.100000
8 Morbius acción 2022 5.200000
9 The Lord of the Rings: The Fellowship of the Ring acción 2001 8.800000
10 Top Gun acción 1986 6.900000
11 Samaritan acción 2022 5.700000

USUARIOS
17
0 Nick1 user1@correo.es 6 7: 2 3 6 7 8 10 15
1 Nick2 user2@correo.es 6 4: 4 6 7 9
2 Nick3 user3@correo.es 5 2: 16 17
3 Nick4 user4@correo.es 7 4: 2 14 15 16
4 Nick5 user5@correo.es 6 3: 6 7 9
5 Nick6 user6@correo.es 7 2: 5 9
6 Nick7 user7@correo.es 7 7: 1 2 3 5 6 8 9
7 Nick8 user8@correo.es 6 4: 2 4 7 9
8 Nick9 user9@correo.es 7 3: 3 5 7
9 Nick10 user10@correo.es 7 3: 2 4 7
10 Nick11 user11@correo.es 6 3: 10 12 13
11 Nick12 user12@correo.es 6 5: 7 8 9 10 11
12 Nick13 user13@correo.es 7 3: 1 4 8
13 Nick14 user14@correo.es 7 2: 2 5
14 Nick15 user15@correo.es 6 2: 4 7
15 Nick16 user16@correo.es 5 3: 2 3 10
16 Nick17 user17@correo.es 8 5: 7 8 9 10 12

OPINIONES
2.000000 8.000000 5.000000 -1.000000 -1.000000 -1.000000 9.000000 -1.000000 -1.000000 5.000000 -1.000000 6.000000
-1.000000 -1.000000 -1.000000 -1.000000 6.000000 -1.000000 8.000000 7.000000 2.000000 -1.000000 3.000000 4.000000
-1.000000 3.000000 2.000000 -1.000000 9.000000 6.000000 8.000000 -1.000000 -1.000000 -1.000000 -1.000000 -1.000000
-1.000000 10.000000 -1.000000 5.000000 4.000000 -1.000000 -1.000000 4.000000 -1.000000 5.000000 2.000000 2.000000
-1.000000 -1.000000 -1.000000 5.000000 -1.000000 8.000000 -1.000000 10.000000 4.000000 2.000000 -1.000000 9.000000
7.000000 6.000000 -1.000000 -1.000000 9.000000 -1.000000 1.000000 3.000000 5.000000 9.000000 -1.000000 -1.000000
1.000000 10.000000 -1.000000 -1.000000 -1.000000 2.000000 -1.000000 9.000000 -1.000000 4.000000 5.000000 5.000000
-1.000000 -1.000000 -1.000000 7.000000 2.000000 -1.000000 5.000000 10.000000 7.000000 4.000000 -1.000000 -1.000000
-1.000000 3.000000 -1.000000 -1.000000 4.000000 -1.000000 -1.000000 9.000000 5.000000 1.000000 8.000000 7.000000
-1.000000 -1.000000 2.000000 6.000000 -1.000000 -1.000000 1.000000 10.000000 8.000000 4.000000 8.000000 -1.000000
-1.000000 -1.000000 -1.000000 -1.000000 6.000000 -1.000000 6.000000 5.000000 -1.000000 3.000000 1.000000 1.000000
2.000000 -1.000000 -1.000000 -1.000000 8.000000 2.000000 8.000000 -1.000000 8.000000 -1.000000 -1.000000 4.000000
6.000000 -1.000000 10.000000 9.000000 -1.000000 9.000000 3.000000 7.000000 -1.000000 -1.000000 -1.000000 9.000000
1.000000 2.000000 3.000000 6.000000 -1.000000 -1.000000 -1.000000 8.000000 -1.000000 -1.000000 6.000000 2.000000
-1.000000 -1.000000 -1.000000 7.000000 5.000000 -1.000000 7.000000 -1.000000 -1.000000 10.000000 9.000000 5.000000
-1.000000 -1.000000 4.000000 -1.000000 9.000000 9.000000 1.000000 -1.000000 8.000000 -1.000000 -1.000000 -1.000000
9.000000 -1.000000 8.000000 2.000000 -1.000000 8.000000 -1.000000 -1.000000 10.000000 7.000000 6.000000 2.000000
Press [Enter] to close the terminal ...
```

Figura 2: Resultado de aplicar el operador « de la clase Opiniones.

5. Main.cpp y estructura de ficheros

En esta sección, comentaremos en detalle cómo debe de implementarse el main.cpp y qué estructura deben de tener los ficheros que almacenan la información del sistema de recomendación a implementar.

5.1. Estructura de main.cpp

Dado que el peso de toda la computación de este programa se realiza en los métodos de las clases, se debe de mantener el fichero main.cpp lo más simple posible. Por tanto, su única función debe de ser la de obtener de argv todos los datos y valores de parámetros, crear un objeto Opiniones que almacene la información de los ficheros indicados y luego llamar a una de las tres funciones que se han creado fuera de la clase Opiniones según el tipo de acción requerida. La acción requerida se indicará mediante un parámetro en la llamada del programa. Recordemos que las 3 funciones a las que hacemos referencia, aunque están localizadas fuera de la clase Opiniones se definen en los ficheros .h y .cpp de la clase Opiniones.

Según el tipo de acción a llevar a cabo, se llamará al programa de alguna de las tres siguientes maneras:

- **Recomendación de películas a un usuario**: Para este caso, la llamada al programa se hará de la siguiente manera:

  ```
  <ejecutable> -a recomendacion -u <idusuario> -n <numusuarios>
  -fu <ficherousuarios> -fo <ficheroopiniones> -fp <ficheropeliculas>
  ```

  Los parámetros podrán aparecer en cualquier orden. Por tanto, los dos ejemplos mostrados a continuación serían opciones de llamada válida y llevarían a cabo la misma ejecución:

  ```
  practicafinal -a recomendacion -u 3 -n 2 -fu usuarios.txt -fo opiniones.txt
  -fp peliculas.txt
  ```

  ```
  practicafinal -a recomendacion -n 2 -u 3 -fo opiniones.txt -fp peliculas.txt
  -fu usuarios.txt
  ```

  Puede verse como lo importante es que detrás de cada opción -<param> aparezca el valor adecuado. Si llamamos al programa utilizando este conjunto de parámetros, el main.cpp debe de recoger los valores de todos los parámetros y, si todo es correcto, llamar a la función generaRecomendacionUsuario. Los tres ficheros indicados se utilizarán para generar un objeto Opiniones con la información contenida en los archivos. **Para mayor orden dentro del programa, todos los ficheros de datos estarán incluidos dentro de un directorio data dentro de la carpeta de proyecto y en las llamadas sólo se indicarán el nombre de los archivos**.

- **Estudio de la valoración de una película**: Para este caso, la llamada al programa se realizará de la siguiente manera:

```
<ejecutable> -a valoracionpelicula -p <nombrepelicula> -fu <ficherousuarios>
-fo <ficheroopiniones> -fp <ficheropeliculas>
```

Algunos ejemplos de llamadas válidas son los siguientes:

```
practicafinal -a valoracionpelicula -p Lord_of_the_Rings -fu usuarios.txt
-fo opiniones.txt -fp peliculas.txt
```

```
practicafinal -p Samaritan -a valoracionpelicula -fu usuarios.txt
-fo opiniones.txt -fp peliculas.txt
```

De forma análoga a la llamada anterior, en este caso el main recogerá toda la información introducida en el vector argv, creará el objeto Opiniones con la información de los ficheros y, finalmente, llamará a la función calculaConsensoPelicula. Pueden utilizarse guiones bajos para indicar más de dos palabras en el título de la película al sistema. Usando la función find y replace de la clase string, es posible sustituirlos por espacios antes de pasar el parámetro a la función calculaConsensoPelicula. Podéis utilizar la web `https://cplusplus.com` para ver en detalle como usar ambos métodos de la clase string.

- **Modificación de opinión de un usuario sobre una película**: Para este caso, la llamada al programa tendrá la siguiente forma:

```
<ejecutable> -a modificacion -p <nombrepelicula> -u <idusuario>
-op <valoropinion> -fu <ficherousuarios> -fo <ficheroopiniones>
-fp <ficheropeliculas>
```

Este tipo de acción llamará a la función cambiaOpinion. Podemos ver un par de ejemplos de llamada a continuación:

```
praticafinal -a modificacion -p Samaritan -u 3 -op 7 -fu usuarios.txt
-fo opiniones.txt -fp peliculas.txt
```

```
praticafinal -a modificacion -p Samaritan -u 3 -op -1 -fu usuarios.txt
-fo opiniones.txt -fp peliculas.txt
```

En la tabla 4, puede verse en detalle qué significa cada uno de los parámetros mencionados en las llamadas a programa. Nótese que la llamada a una u otra función depende principalmente de la cadena introducida en el parámetro -a. Si la cadena introducida en este parámetro no se reconoce, entonces el programa acaba con un mensaje de error.

En caso de haber un error en la indicación de los parámetros, se puede pensar que se podrían pedir los parámetros uno a uno mediante la directiva cin. Sin embargo, al ser estos demasiados, no es recomendable.

-a	Acción a llevar a cabo. Puede ser recomendar película, estudiar la valoración de una película o modificar un valor de opinión.
-u	Id del usuario.
-n	Número de usuarios afines buscados.
-fu	Fichero que contiene la información de los usuarios.
-fo	Fichero que contiene la información de las opiniones.
-fp	Fichero que contiene la información de las películas.
-p	Nombre de la película.
-op	Valoración de la película. Puede ser -1 para indicar que queremos eliminar una valoración del sistema.

Tabla 4: Significado de los parámetros de llamada del programa.

Por tanto, no pediremos que se implemente nada que solucione este problema. Sólo será necesario indicar un mensaje de error.

5.2. Estructura de los ficheros del sistema de recomendaciones

En esta sección pasaremos a detallar la estructura que deben de tener los ficheros con los que trabaja el sistema de recomendaciones. Junto con esta práctica se proporciona un archivo .zip que contiene 3 ficheros ejemplo de forma que puedan servir al alumno para hacer pruebas de las funciones y métodos que se deben de implementar en esta práctica. Es importante que los ficheros se almacenen en el directorio ./data/ del proyecto y que en los parámetros del programa sólo se indique el nombre del fichero. De esta forma, no habrá problemas en la lectura y el proyecto creado será portable lo que permitirá al profesor corregirlo adecuadamente. La estructura de los ficheros puede verse en las figuras 4, 5 y 3.

En el caso del fichero de las películas, puede verse como se ha mantenido el formato original que utilizamos en la práctica 2. Esto permitirá poder mantener el código de lectura de fichero que se implementó entonces. En el ejemplo proporcionado, el número de películas a leer es bastante pequeño. Esto se ha diseñado así para que a la hora de hacer pruebas, sea más fácil identificar errores. Aun así, el programa final debe de ser capaz de ser ejecutado con cualquier número de películas.

En cuanto al fichero de usuarios, se ha mantenido el formato utilizado en la impresión de datos por pantalla que se utilizó en la práctica 4. Por tanto, es interesante la implementación de un operador » complementario al operador « que permita leer los datos necesarios para generar el objeto ConjuntoUsuarios asociado al objeto Opiniones. El primer elemento del archivo de usuarios es el número de usuarios, lo cual permite identificar cuantos usuarios hay en el fichero. Cada usuario, como puede verse, se almacena en una línea distinta.

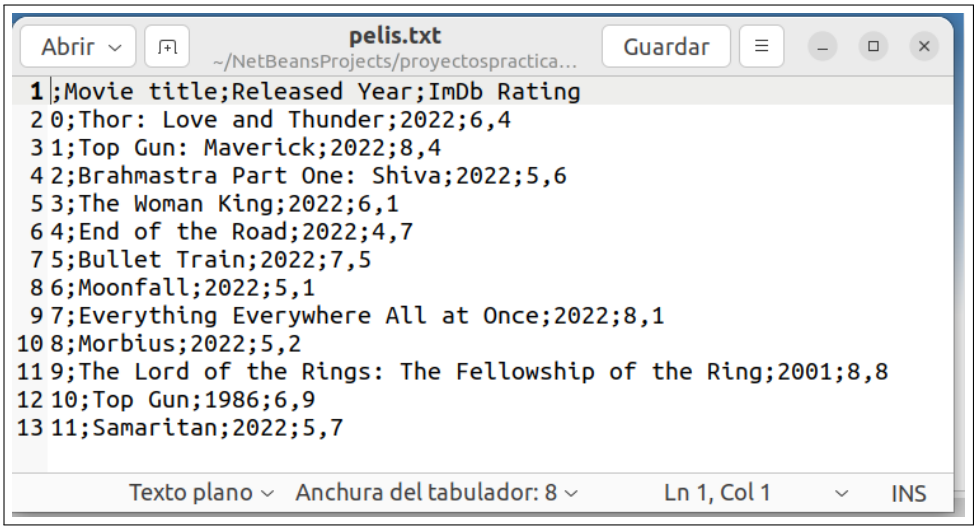

Figura 3: Ejemplo de fichero donde se almacena la información de las películas.

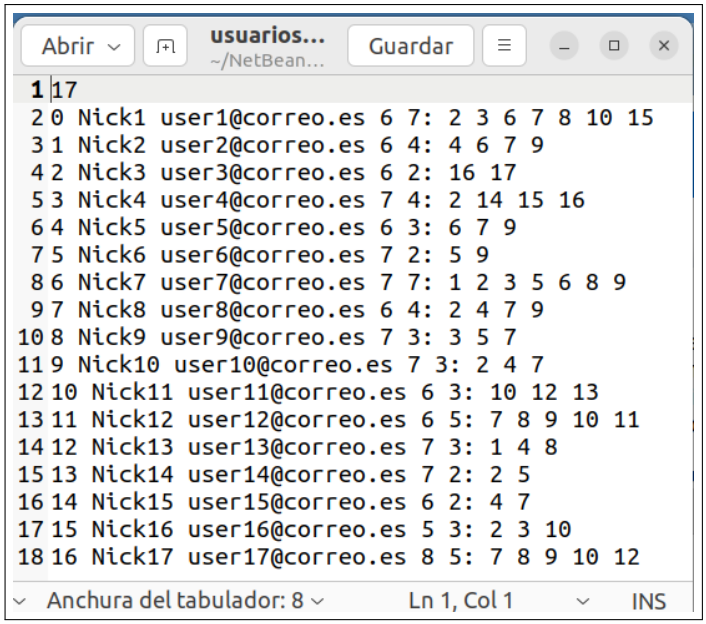

Figura 4: Ejemplo de fichero donde se almacena la información de los usuarios.

Por último, puede verse como en el fichero de opiniones se almacenan primero dos valores, el tamaño de filas y columnas de la matriz. A continuación, a partir de la siguiente línea, se almacenan todos los valores de la matriz de forma que cada fila de la matriz aparece en una línea de fichero diferente. Los datos dentro de cada fila están separados por espacios.

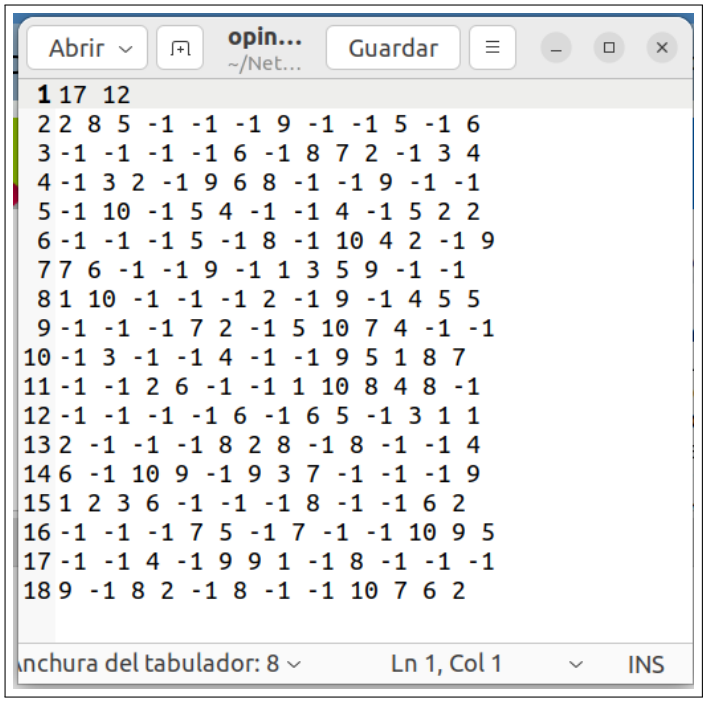

Figura 5: Ejemplo de fichero donde se almacenan las opiniones de los usuarios sobre las películas.

6. Detalles a tener en cuenta

Por último, a modo aclaratorio, en esta sección se indican varias propiedades que el programa implementado debe cumplir:

- El programa requiere de la introducción de parámetros para funcionar. En caso de que los parámetros no se introduzcan se dará un mensaje de error.
- En el caso del proceso de modificación de opiniones, una vez terminada la modificación deben sobreescribirse los ficheros que se leyeron con la nueva información.
- Todos los ficheros deben de estar localizados dentro del directorio data del proyecto.
- El alumno puede añadir todas los métodos, funciones y operadores extra que considere oportunos.
- En esta práctica no se proporciona ninguna carpeta de proyecto inicial. El alumno debe de trabajar utilizando las clases implementadas en las prácticas anteriores y generando **una única carpeta de proyecto** que incluya las cinco clases indicadas y el fichero main.cpp. En esta práctica final se valora únicamente la funcionalidad nueva introducida por lo que, de ir el alumno con retraso en la realización de las anteriores prácticas, puede centrarse en implementar, de las prácticas 2 y 4, únicamente aquello que vaya a utilizarse directamente en el sistema de recomendaciones planteado. La no imple-

mentación de métodos que no se usen directamente en esta práctica no conlleva bajada de nota en la evaluación de este proyecto final.

7. Ejemplo de ejecución

Con el objetivo de que el alumno pueda entender mejor la implementación del programa, en la siguiente sección se muestran varios ejemplos de ejecución del programa, una para cada una de las tres posibles acciones a realizar. En el primero, recomendamos películas a un usuario, en el segundo consultamos la media y consenso de una película y por último, en el tercero, modificamos una valoración de usuario.

7.1. Generación de recomendaciones de películas

En la Figura 6, podemos ver el resultado de la ejecución de la acción recomendación sobre el usuario con id 2 (-u 2). En el parámetro n hemos indicado que queremos realizar el proceso de recomendación a partir de los dos usuarios más similares (-n 2).

```
Recomendamos al usuario Nick3 las siguientes peliculas:
7 Everything Everywhere All at Once acción 2022 8.100000
10 Top Gun acción 1986 6.900000
3 The Woman King acción 2022 6.100000
8 Morbius acción 2022 5.200000
6 Moonfall acción 2022 5.100000
4 End of the Road acción 2022 4.700000

Press [Enter] to close the terminal ...
```

Figura 6: Resultado de la ejecución de la acción de recomendación de películas.

7.2. Estudio de la valoración de una película

En este caso, analizamos la valoración de la película "The Lord of the Rings: The Fellowship of the Ring". Por tanto, indicamos el nombre de la película o parte de él en el parámetro -p y ejecutamos la acción valoracionpelicula. El resultado puede verse en la Figura 7.

```
Valoración de: The Lord of the Rings: The Fellowship of the Ring
Los usuarios dan una valoración media de 5.25
Esa valoración tiene un consenso de:0.337879
El consenso global de los usuarios en la plataforma es de:0.331484
Press [Enter] to close the terminal ...
```

Figura 7: Resultado de la ejecución de la acción de estudio de la valoración de una película.

7.3. Modificación de una opinión de usuario

En este caso, no haría falta imprimir nada por pantalla. Si se desea, se puede imprimir por pantalla toda la información de la clase con la modificación realizada usando el operador « de la clase Opiniones tal y como muestra la Figura 2.

8. Entrega

La entrega consistirá en un fichero comprimido que contendrá la carpeta del proyecto con todas las clases implementadas. En esta práctica no se proporciona código asociado, el alumno debe implementar el proyecto incluyendo los archivos ya desarrollados en prácticas anteriores y creando la clase Opiniones desde cero.

La fecha de entrega de la práctica será la indicada en PRADO. **NO SE PERMITEN ENTREGAS FUERA DE FECHA**.

Se recuerda que es muy importante que el proyecto **compile perfectamente** ya que, de no hacerlo, la entrega no será válida.